3D Euler cache: equivalence and a scoped optimum

A staged investigation of optimal solvers within a restricted class.

Current result: Lean checks exact-arithmetic cache equivalence, conservation, and minimum thermodynamic-call cost within a fixed schedule interface. CPU timing separately shows a 1.05–1.25× one-step speedup. **Optimal time to solve the Euler PDE at a given accuracy remains open.**

The fixed problem

The investigation improves the implementation of a fixed three-dimensional fluid discretization and now checks a model of that change formally. It does not search over numerical methods or establish the fastest route to a given PDE accuracy. The proof, executable correspondence checks, and hardware measurements remain separate evidence layers.

Item	Fixed contract
Equations	3D compressible Euler; ideal gas; gamma = 2; density, three momenta, total energy
Discretization	Uniform periodic Cartesian grid; first-order shared-face Rusanov flux; forward Euler
Candidate change	Compute pressure and sound speed once from the old state; reuse them for x, y and z fluxes
Output requirement	Same complete discrete update for the same state and supplied timestep
Measured objective	CPU time per full update kernel, including cache and output allocation

What has and has not been proved

The formal result is over real-valued five-component states, arbitrary periodic neighbor permutations, and supplied timestep/grid ratios. The cache must be fresh. Neither positivity after a step nor PDE convergence follows from equality with the original scheme. Python execution, floating-point rounding, and compiler behavior are not formally refined to the Lean model.

The Lean-checked result

Fifteen named theorems passed compilation and an independent transitive-axiom audit in Simjecture's bounded capability sandbox. Their only recorded axioms are propext, Classical.choice, and Quot.sound. Both model files, generated audits, logs, hashes, and job receipts are retained under formal/record/.

Exact-arithmetic equivalence

EulerCache.lean defines pressure, sound speed, physical flux, the spectral bound, Rusanov face flux, and both three-axis updates. Fresh-cache substitution proves equality at every cell and component. A second theorem proves equality for every finite number of fixed-ratio steps with a fresh cache each step. The concrete cached update conserves all five component totals on finite periodic grids.

Restricted schedule model

Eval(i) creates a thermodynamic pair from the immutable old state. Use(a,i) requires that pair to be present. Drop(i) evicts it. The cache starts empty, and all three directions at every cell must be used. Only Eval creates data: algebraic bypasses, half-pair caching, cross-cell reuse, warm starts, and changed numerical methods are excluded. Availability is modeled separately from the fresh-value condition in the equivalence proof.

Quantity	Reference	Cached
Thermodynamic evaluations	3N	N
Directional Uses (single pass)	3N	3N
Weighted library cost	(3t + 3f)N	(t + 3f)N

Every consumed cell must first be evaluated. Counting distinct evaluated cells yields the lower bound N; the cached schedule attains it. With nonnegative integer costs t per Eval, f per Use, and d per Drop, it minimizes weighted cost among legal, complete schedules with exactly 3N Uses. The saving is 2tN, strictly positive when t>0 and N>0. The unchanged face/update work is not counted; adding an identical common cost preserves the comparison.

Connection to NumPy

Instrumentation observes 3N versus N pressure/square-root evaluations at 3^3, 8^3, and 16^3. The scalar transcription of the Lean formula matches the tested NumPy update (maximum difference 0.0e+00). Deliberately stale pressure changes the result by 1.276e-03; omitting an Eval is rejected by the schedule interpreter. These are empirical correspondence checks, not a compiler or floating-point proof.

Measured evidence

One warm-up call per kernel; 11 timing trials per grid; deterministically shuffled execution order. Times include cache creation and returned-array allocation. Timestep selection and validation are outside the timing interval.

Grid	Reference (ms)	Cached (ms)	Ratio
16^3	0.800	0.686	1.166×
32^3	8.699	8.317	1.046×
48^3	25.073	20.848	1.203×
64^3	52.537	42.064	1.249×

Update and admissibility checks

Input	Grid / steps	Equality	Minimum rho / p
random	8^3 / 8	Bitwise identical	0.8147 / 1.0602
random	12^3 / 8	Bitwise identical	0.7582 / 0.9455
coupled	16^3 / 20	Bitwise identical	0.9190 / 0.9441
blast	24^3 / 20	Bitwise identical	0.8734 / 0.1000

All checked steps retained positive density and pressure. The largest absolute drift of a volume-weighted conserved total was 1.66e-14. The 3N-to-N evaluation reduction is now instrumented, and N is proved minimal within the restricted schedule interface.

What these measurements do not establish

The random and coupled cases exercise nonlinear three-dimensional states. The spherical pressure blast is an execution/admissibility check without an independently validated shock solution. There is no convergence result for these cases. The real-valued all-state model equivalence is separately proved; these finite tests do not establish IEEE floating-point equivalence for all inputs.

The timing ratio compares two NumPy implementations on one CPU host. It does not prove a best constant, end-to-end speedup, GPU advantage, or optimal time to a prescribed error. The roughly 1.05× result at 32^3 is small enough to warrant repetition under controlled host load.

Linux-6.18.54.1-microsoft-standard-WSL2-x86_64-with-glibc2.39

Host: AMD Ryzen 9 9900X. Python 3.12.13; NumPy 2.5.1. This experiment uses CPU execution.

Proof gates and replay

Gate	Required evidence	Status
1. Preserve the update	Frozen source, nonlinear 3D trajectories, conservation/admissibility checks, all timing trials	Empirically passed
2. Formalize implementation	Actual Euler flux; admissibility assumptions; exact-arithmetic cache equivalence; allowed dataflow transformations; meaningful cost lower bound	Proved in scoped model
3. Solve to an error target	Manufactured smooth solutions, independent shock references, refinement studies, all timesteps and setup costs	Open
4. Discover broader methods	New flux/reconstruction/time-integrator candidates, certified counterexamples, matched-accuracy baselines, new lower bounds	Open-ended

The achieved optimum concerns caching schedules in a fixed kernel interface. It is a dependency lower bound, not a lower bound for every possible Euler algorithm. Nonuniform memory costs, cache capacity, algebraic transformations outside the interface, numerical accuracy, and total simulation work require further analysis.

Reproduce locally

From the repository root, run stage 1 with optimization/run_local.py. For the new proof and instrumentation jobs:

.venv/bin/python demos/lean_euler/optimization/formal/run_local.py

--output artifacts/lean-euler/formal-replay-001

Choose a new output directory. The formal driver submits two exploratory lab jobs, freezes the Lean/Python sources, and retains formal-results.json, instrumentation.json and receipts. The records distinguish proved, empirical and open claims. This is assistant-authored scripted formalization without independent scientific review or autonomous algorithm discovery.

Sources and provenance

The candidate and measured evidence are in PR #90: github.com/tomzhu0225/simjecture/pull/90. The PR is draft while the 3D research develops.

[1] X. Zhang and C.-W. Shu (2010), positivity-preserving Euler schemes on rectangular meshes. This provides established context for separate admissibility arguments, not an optimality proof for this candidate. Author-hosted paper.

[2] Clawpack, Riemann solvers: directional interface solvers in multidimensional methods. Official documentation.

experiment.py SHA256:
b70d8a26ca230938e1bd8c14ee2ff478472cf19fe8609825b73a76bac9833856

euler.py SHA256:
505ebff19d161561f3723a259e65e9476e7c0b40c357146df905dfa56e95d2c2